



Computational Methods in Particle Physics

Part 3: Machine learning

Drs Ben Hodkinson & Holly Pacey
Oxford HEP DPhil Programme
Hilary Term 2025

Introduction

What we will cover:

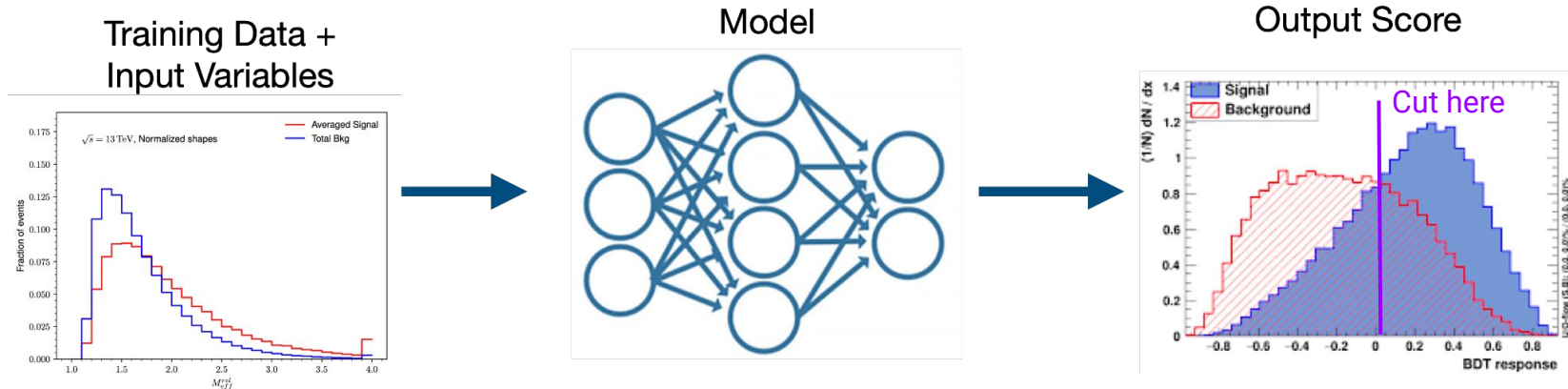
- An overview of common-to-HEP Machine Learning algorithms
- Practical tips on how to use these effectively
- Common questions/issues
- There's not time to cover everything but will provide links to useful tutorials and resources if you'd like to learn more.



Useful links

- HEP Software Foundation Machine Learning tutorial:
 - <https://hsf-training.github.io/hsf-training-ml-webpage/index.html>
 - Code + video – covers basic theory + examples, ~few hours total
- Keras tutorial (Neural Networks):
https://github.com/TomStevo/GRADnetDMKeras2024/blob/main/GRADnet_2024_keras_ML.ipynb
- [ONNX](#) ([github](#))
 - Common format for ML models across frameworks and tools
- HEP ML living review <https://iml-wg.github.io/HEPML-LivingReview/>
- LBNL ML summer school:
 - <https://indico.physics.lbl.gov/event/2850/timetable/#20240812>
 - <https://github.com/ml4fp/2024-lbnl>
- CERN also offers ML summer schools
 - Last iteration in 2023: <https://indico.cern.ch/event/1229514/>

What is Machine Learning?



Two main general use cases:

- **Classification:** Separating different classes of data (i.e. signal vs background, particle ID, jet tagging) → outputs are classification scores
- **Regression:** Computing underlying variables (e.g. mass) → outputs are estimations of the variable values

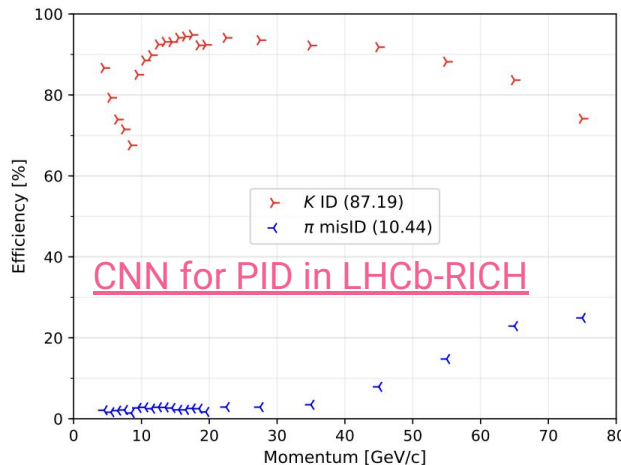
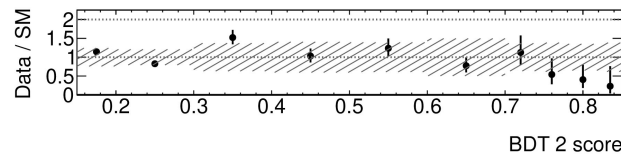
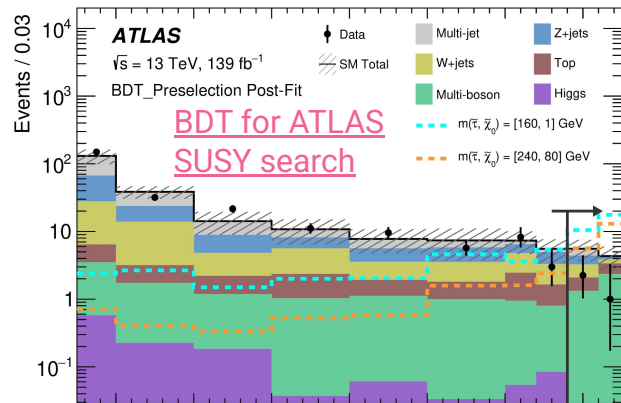
Typical HEP applications

Analysis Applications

- Signal vs background classification
- Matching objects to common parent/decay chain

Performance Applications

- Simulation, Reconstruction
- Tagging (classifiers)
 - Particle ID
 - Jet-tagging (b-jet / W / Z / QCD / top)
- Calibration (regression)
 - Map detector-level quantities to truth-level



Why you might want to use ML

- **Performance:** ML should always do better than or, in the worst case, just as well as a 'Cut and Count' based selection
- HEP is a great **ML laboratory** - typically have large amount of clean, high quality data
- It's something you as a student can have quite a lot of **agency/control** over
- Though it might seem intimidating at first, it's actually **relatively easy to do** - tools are easier to use than ever before (you also have LLMs to help you)
- It's good to have on your **CV** (for whatever you do next)
- It's an evolving field, lots of **possibility for new ideas/applications!**
- It's fun!

Why you might NOT want to use ML

- Is it really necessary for your problem?
- Takes time to do and review
- Risk of bugs/errors that are hidden by your black box model and invalidate your results
- Need lots of training events and computing resources
- Risk of over-tuning ML model / analysis to a specific signal model/simulation
 - Eg. Specific MC generator or bSM scenario
- Extra steps for re-interpreting a collider search
- It's not a magic wand and is not guaranteed to provide a step change in performance



Common features of ML algorithms

- Essentially a function of the input features with a very large number of parameters
- Parameters adjusted during **training** to minimise a **loss function**
- **Supervised learning**: training data is labelled
 - Eg. classification and regression
- **Unsupervised learning**: training data is not labelled
 - Eg. clustering, anomaly detection, generative models



Common loss functions

Classification: **Categorical cross-entropy** $\mathcal{L}_{\text{CCE}} = \sum_C t_C \log(P_C)$

- t_C = truth-label for class C
- P_C = output score for class C

Regression:

- **Mean squared error**
 - Heavily punishes outliers with larger errors

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

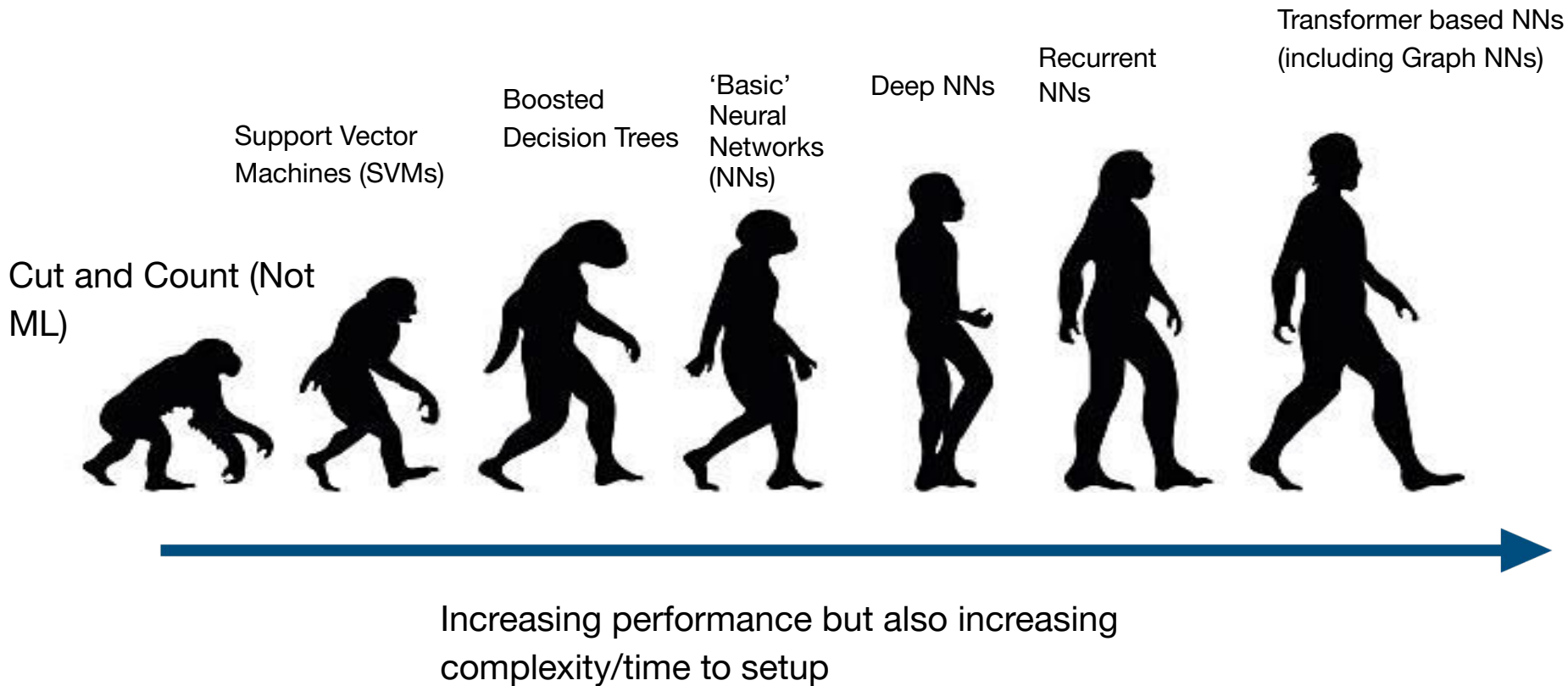
- **Mean absolute error**
 - Focus less on outliers

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **Huber**
 - Middle ground
 - Typical $\delta=1.5$

$$\mathcal{L}_{\text{Huber}} = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & , \quad |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & , \quad \text{otherwise} \end{cases}$$

Common ML algorithms



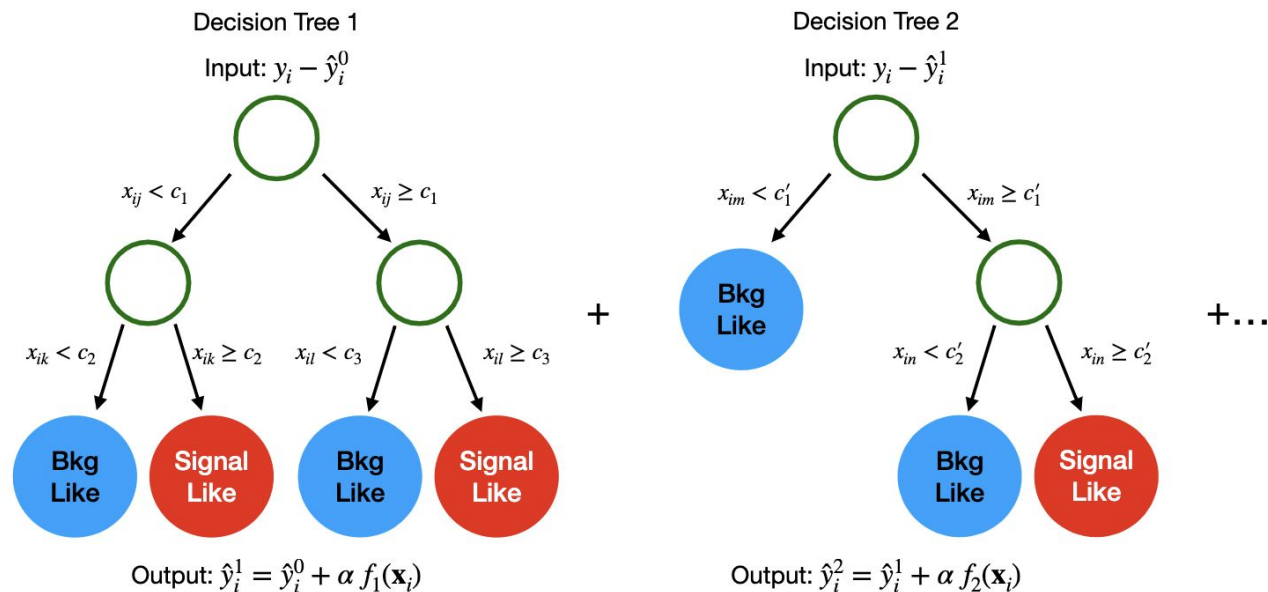
A brief explanation of Boosted Decision Trees (BDTs)

Decision tree: Series of cuts on set of input features $\mathbf{x}$ to produce high-purity signal and background 'leaves'.

- Classifier scans through all features at each node to determine cut that maximises separation.
- User-defined stopping criteria (eg. max. num decisions, min. sample size in each node)

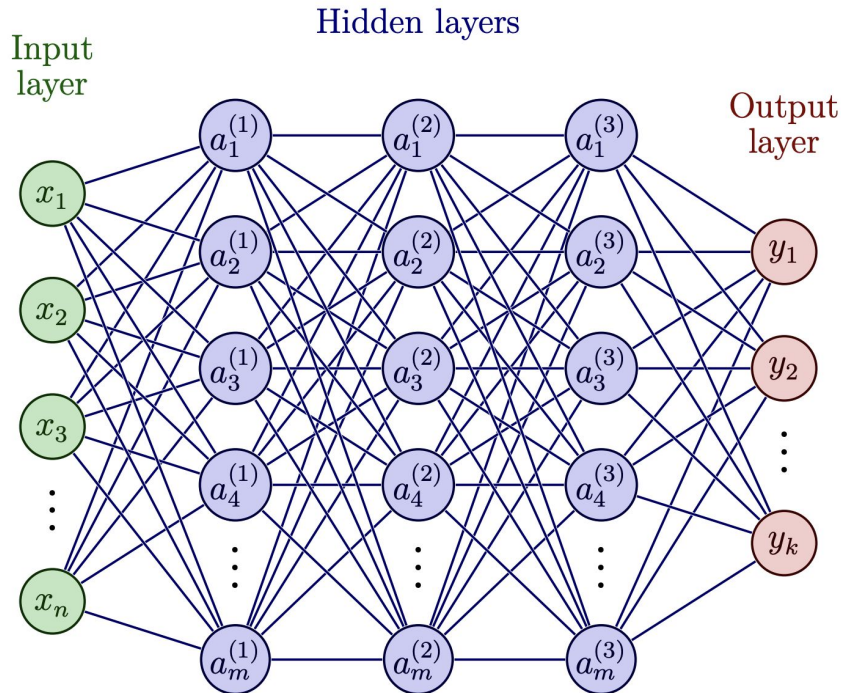
BDT: Series of decision trees ("forest"), each trained on the residuals of the previous tree, correcting for errors

- Misclassified events are up-weighted
- Final score is weighted average across all trees in the forest
- Different to random forests, where trees are trained in parallel)



[StatQuest youtube video on Xgboost](#) (math details)

A Brief Explanation of Neural Networks



Transformation at each node:

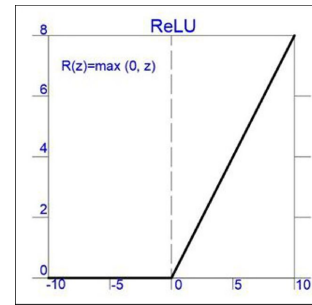
$$a_j^{(i)} = \sigma \left(\sum_{k=1}^m w_{j,k} \cdot a_k^{(i-1)} + b_j^{(i-1)} \right)$$

Weights + bias term

- Adjusted during training via “gradient descent” to minimise loss function

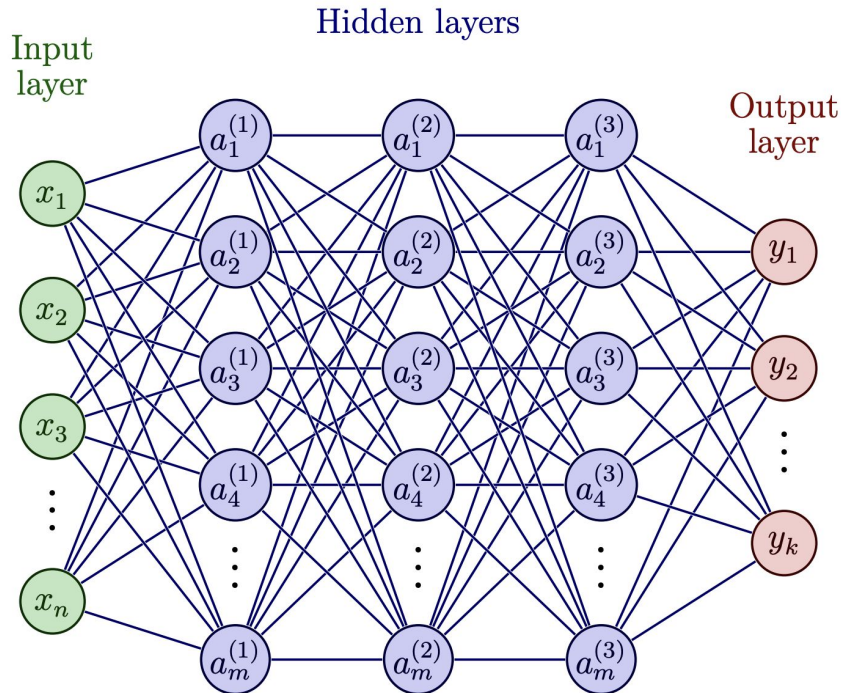
Activation function

- User-defined
- Introduces non-linearity



[3blue1brown youtube playlist on NNs](#)

A Brief Explanation of Neural Networks



Transformation at each node:

$$a_j^{(i)} = \sigma \left(\sum_{k=1}^m w_{j,k} \cdot a_k^{(i-1)} + b_j^{(i-1)} \right)$$

Weight update for training step t:

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$$

“Learning rate” $\leftarrow \eta$

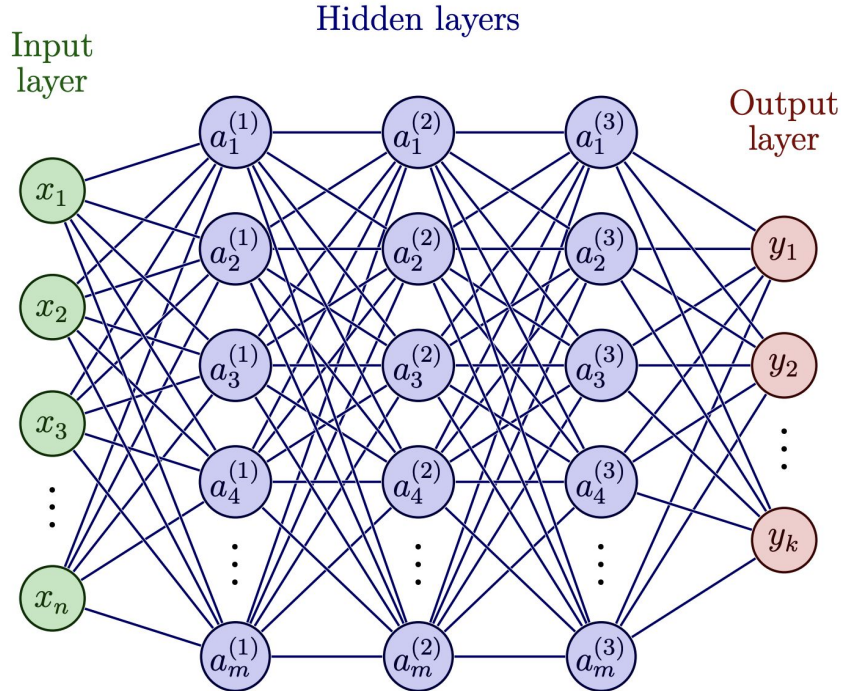
Gradient of loss function $\leftarrow \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$
→ determined via “backpropagation”

Variations on standard gradient descent:

- Stochastic Gradient Descent
- Adaptive Learning Rates
- Adam

[3blue1brown youtube playlist on NNs](#)

A Brief Explanation of Neural Networks



Universal Approximation Theorem

A feed-forward neural network with a single hidden layer and finite number of neurons can approximate any real continuous function arbitrarily well given an appropriate activation function

→ But may need an arbitrarily large number of neurons or training data points

[3blue1brown youtube playlist on NNs](#)

Assessing Model Performance - Metrics

- What's important for your problem?
- For classification:
 - ROC Curves (Receiver Operating Characteristic)
 - Precision and Recall → can be more useful for imbalanced classes
- For Regression:
 - Mean Absolute error, Root Mean Square error etc.

Precision

Of all **positive predictions**,
how many are **really positive**?

$$\frac{TP}{TP + FP}$$

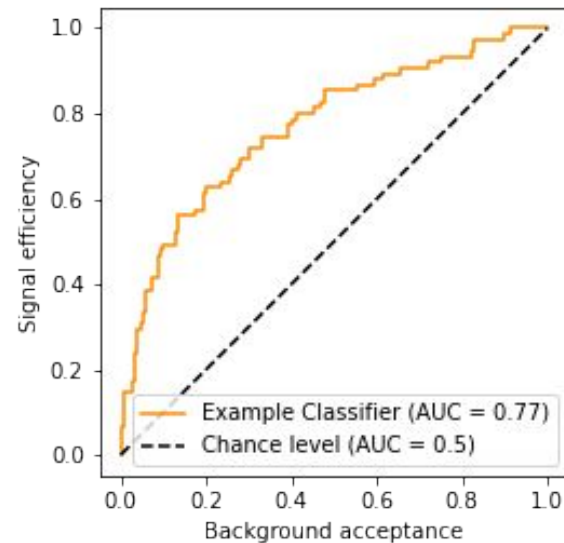
		Real Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

Recall

Of all **real positive cases**,
how many are **predicted positive**?

$$\frac{TP}{TP + FN}$$

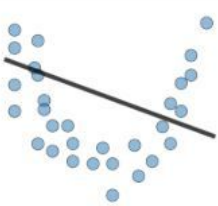
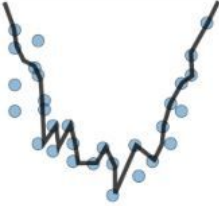
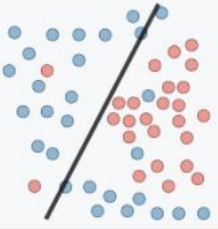
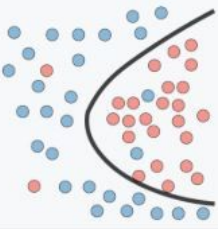
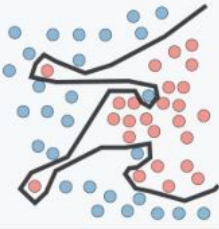
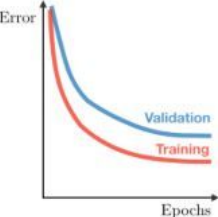
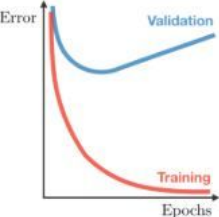
		Real Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN



AUC = Area Under Curve

Overtraining

- We care about performance on data **not seen** during the model training (ie. the “test” set)
- Check for overfitting by comparing performance metrics on “training” and “testing” datasets
 - Usually expect training performance to be a bit better than test set performance
 - Significant difference might indicate mode that the assessment of performance on unseen data is unreliable
- Can also plot distributions of output and perform a statistical test
- Causes:
 - Too many degrees of freedom in architecture
 - Not enough training data
 - Lack of discriminating features

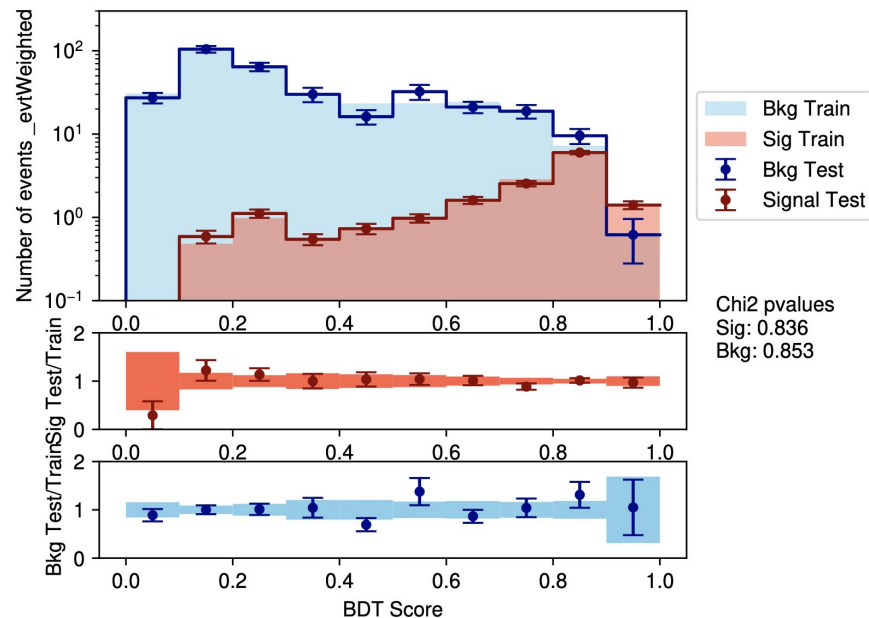
	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Regression illustration			
Classification illustration			
Deep learning illustration			

From

<https://www.kaggle.com/discussions/getting-started/166897>

Overtraining

- We care about performance on data **not seen** during the model training (ie. the “test” set)
- Check for overfitting by comparing performance metrics on “training” and “testing” datasets
 - Usually expect training performance to be a bit better than test set performance
 - Significant difference might indicate mode that the assessment of performance on unseen data is unreliable
- Can also plot distributions of output and perform a statistical test
- Causes:
 - Too many degrees of freedom in architecture
 - Not enough training data
 - Lack of discriminating features

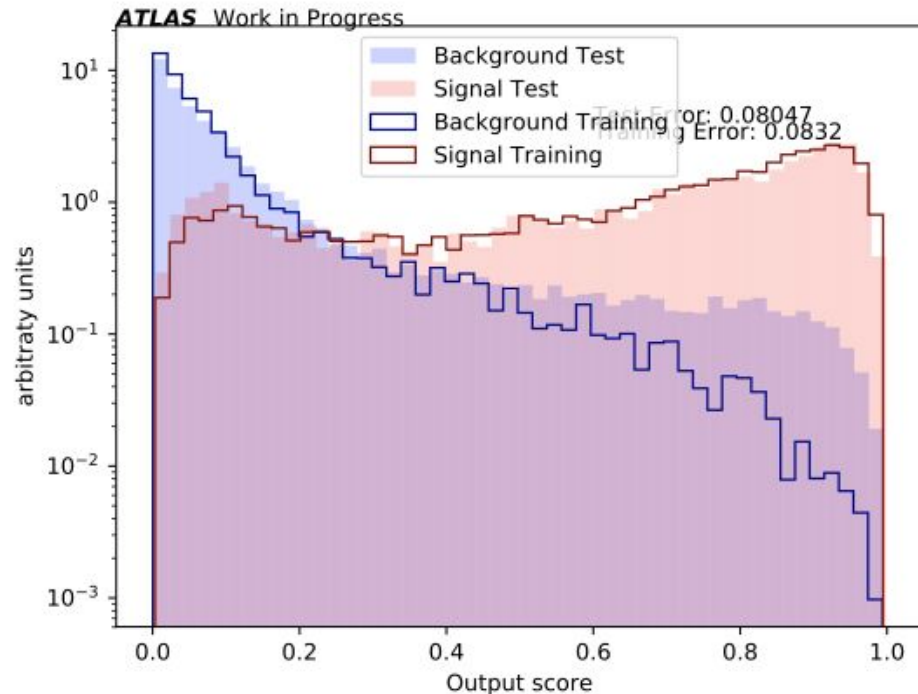


Train vs. validation vs. test data

- Common practice to divide the data set into training/validation/testing data
- **Training set** → used to train your model
- **Validation set** → used to monitor overfitting during training
- **Test set** → independent dataset used to evaluate model performance after training

Note on comparing test/train distributions

- Pay attention to the tails as these are often the regions we care about!
- Eg. KS test might not be sensitive to the train vs. test discrepancy in the background tail here:



Cross Validation

- Final model evaluation should not occur on data seen in training
- But often in PP we can't afford to "throw away" a large portion of simulated data for training (increases stat u/c of evaluation)
- K-Fold cross validation
 - Split data into folds and train multiple models (each with same hyperparameters, input features etc)
 - For final evaluation, each data point is evaluated using a model for which it was not part of the training

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Model 1	Evaluation	Training	Training	Training	Training
Model 2	Training	Evaluation	Training	Training	Training
Model 3	Training	Training	Evaluation	Training	Training
Model 4	Training	Training	Training	Evaluation	Training
Model 5	Training	Training	Training	Training	Evaluation

Usually split events by event number to ensure for MC that folds are consistent across systematic uncertainties - but ideally use a different splitting method during Hyperparameter tuning to avoid over-tuning to the evaluation set

Cross Validation with hold out validation set

Ideally, have **two different non-training sets per fold**

Use extra validation set for e.g.
Early stopping

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Model 1	Evaluation	Training	Training	Training	Testing
Model 2	Testing	Evaluation	Training	Training	Training
Model 3	Training	Testing	Evaluation	Training	Training
Model 4	Training	Training	Testing	Evaluation	Training
Model 5	Training	Training	Training	Testing	Evaluation

Train/Test Split size or how many folds?

- Larger training set = can train more complex (and thus better performing) models (without overfitting)
- Larger test set = more precise/reliable estimate of model's performance
- Typical train:test ratios range from 1:1 to 10:1 (likely diminishing returns for bigger splits in any case)

Pre-processing

- **Handle NaN inputs** → remove or set to numerical default value)
- **Zero-padding** → eg. input features include electron pT but some events have no electrons)
- **Standardise input features**
 - Ensures features have a common scale
 - $X' = (X - \mu)/\sigma \rightarrow$ normalize
 - Alternative: Scale to [0,1] (preserve 0's, better if σ is small)
 - Most packages have tools to do this for you (eg. [scikitlearn](#), [keras](#))
- **Normalise targets**
 - Not essential, but may help in some regression tasks
- **Phi-invariant detector (eg. ATLAS)? → Rotate events to a common phi angle**
 - NN doesn't need to learn phi-invariance of the detector
 - Increases statistical power of training set
 - Only if you're using angular information
- **Calculate sample weights if needed**
 - Eg. Weights to flatten target distribution to reduce bias

→ **Post-processing: Undo any pre-processing of the targets!**

Pre-processing - Imbalanced Classes

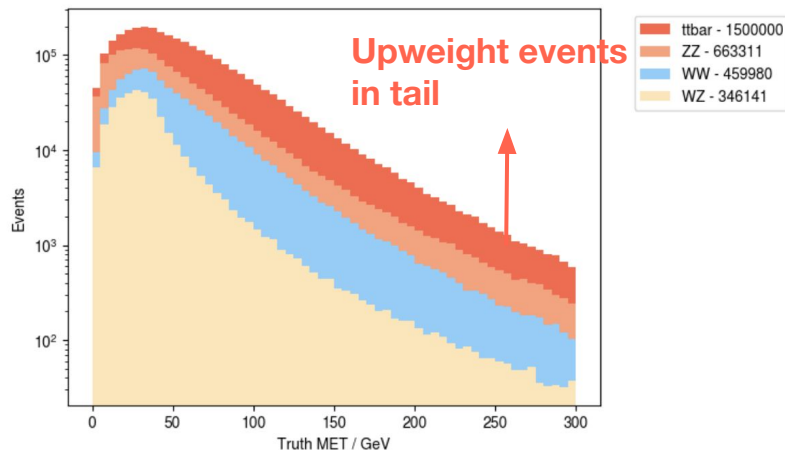
- Classification problems will often involve imbalanced classes
 - e.g. typically a lot more background than signal
- Depending on loss function, model can achieve good performance simply by classifying everything as the majority class
 - Symptom of this can be having a very narrow range of output scores (typically between 0 and 1)
- Usual way around this is to **re-balance** - i.e. apply additional weight(s) so the sum weights of each class are equal
- N.B. be aware you have two different kinds of **imbalances - both weighted and unweighted** - ideally you want ~similar amounts of raw statistics for each class
- Also consider the impact of preselection - balance will depend on phase space

Pre-processing - sample weights

- Most **modern ML packages** support the use of **sample weights**

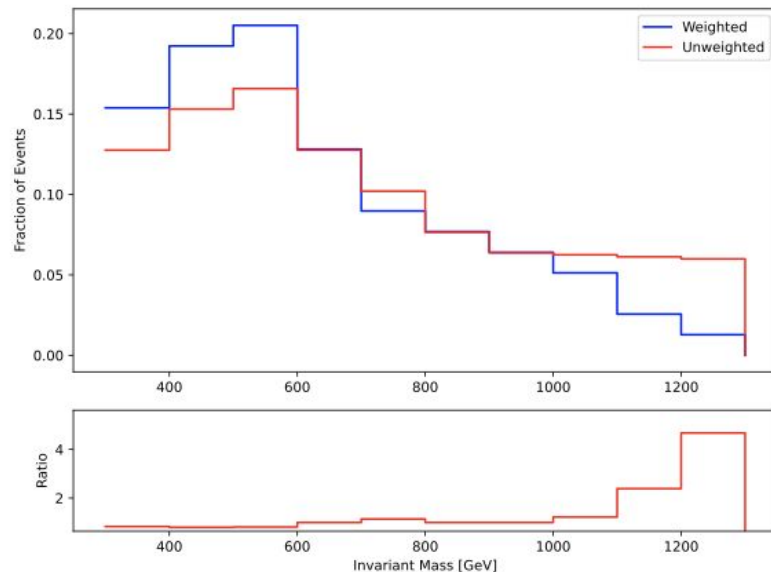
- Up-weight low-statistics regions
- Use MC weights to more accurately represent the underlying distribution

- Contribution of each data point to the loss function is weighted



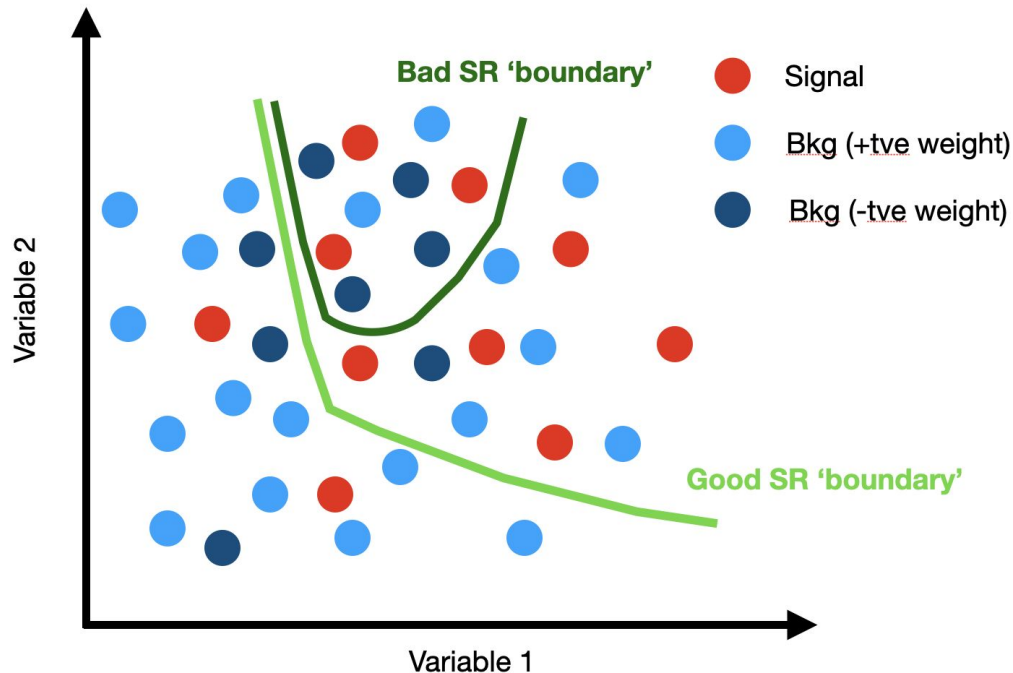
$$L(y_i, p_i) \propto \sum_{i=1}^N -y_i \ln p_i + (1 - y_i) \ln(1 - p_i),$$

$$L(y_i, p_i) \propto \sum_{i=1} w_i [-y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$



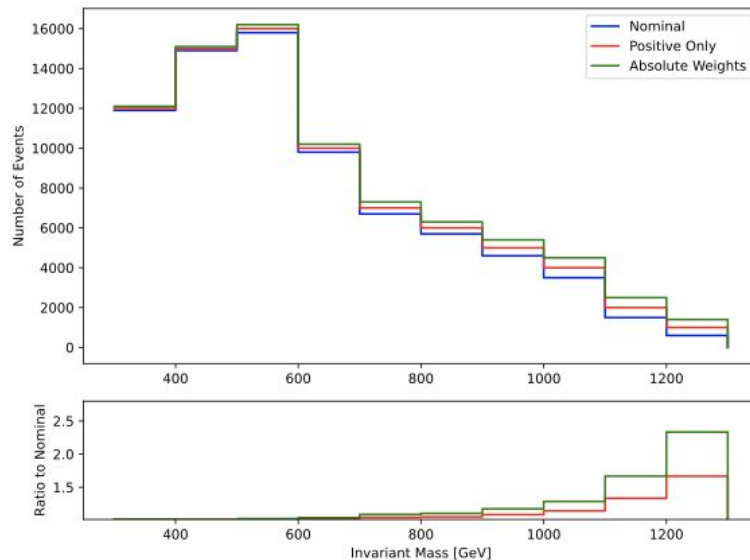
Negative MC weights

- What should you do with negatively weighted events?
- For MC, these can arise from e.g. NLO corrections but can also get neg. weights for data-driven bkg
- Issue is that negative event yields are obviously unphysical - **negatively weighted events shouldn't be separated from regions of phase space with excess of positively weighted events they are supposed to cancel with.**
- But e.g. for binary classification, ML model is incentivised to do exactly that (i.e. separate neg weighted bkg events from rest of the bkg and classify them as signal)



Negatively Weighted Events - what to do

- Common strategies:
 - **Don't include them in the training** (safe as avoids deliberate bias but reduces stats and changes shapes, but can reweight input distributions so shapes match)
 - **Include them but set all weights to be positive** (further distorts shapes, but retains stats, again can reweight distributions)
 - For most cases this will probably make performance worse so personally wouldn't recommend
 - **Restrict granularity** with which model looks at data (through e.g. BDT hyperparameters), but this can also impact performance
- How much of a problem/which strategy is best depends on samples/phase space your using (be aware that some MC samples have ~40% negative weight fraction)
- Also to be clear these **only apply to the training** - when evaluating/doing the actual analysis you have to include the weights as they are!



Note: Hopefully one day this won't be a problem - [soft cell resampling](#)

Hyperparameters

- Aspects of ML models not related to input data are controlled by hyperparameters
- Need to be tuned to your specific dataset/problem
- Exact names and details of HPs depends on software package
- Impact on performance can vary
 - Don't necessarily need to find the most optimal set but need to be in the right ballpark
 - Can have large impact!

HP tuning - BDTs

XGBoost Hyperparameters

Hyperparameter	Description
learning_rate (eta)	Controls the step size at each iteration to prevent overfitting. Lower value requires more trees.
n_estimators	Number of boosting rounds or trees to build. Too many may lead to overfitting.
max_depth	Maximum depth of a tree. Increasing this value makes the model more complex.
min_child_weight	Minimum sum of instance weight needed in a child. Higher values prevent learning specific relations.
subsample	Fraction of training data sampled for each tree. Prevents overfitting, but too low can cause underfitting.
colsample_bytree	Fraction of features sampled for each tree. Similar to max_features in RandomForest.
gamma (min_split_loss)	Minimum loss reduction required for a further partition on a leaf node. Higher values make the model conservative.
reg_lambda	L2 regularization term on weights. Increasing it makes the model more conservative.
reg_alpha	L1 regularization term on weights. Helps in preventing overfitting.

Reminder: want best performance on unseen data without significant over-fitting

Tips

- Balancing NTrees vs learning
 - Smaller learning rate will need larger Ntrees to get same performance
- Parameters which can reduce overfitting
 - Max Depth, Min number of entries per leaf node, number of cut boundaries
- Reasonable place to start:
 - Shallow trees (depth around 4)
 - Ntrees ~few hundred
 - Learning rate ~0.1
- Grid search over HPs can be useful for optimization and to get idea of impact

HP tuning for NNs

- **Loss function**

- Depends on the problem
- Regression: MSE vs. MAE vs. Huber → depends on how you want to focus on outliers
- Classification: Cross-entropy
- “Optimal transport” losses (eg. Sinkhorn)

- **Learning rate**

- Too large = “overshoot” the global loss minimum
- Too small = get “stuck” in local loss minima
- Reasonable values to try initially: 0.0001 / 0.001 / 0.01 / 0.1

- **Optimizer (gradient descent method) → Adam, SGD, etc.**

- **Number of hidden layers and nodes**

- Too large = overtraining and slow/costly training
- Too small = fast training but can’t encode all the information / sub-optimal performance
- More training data and more computational resources = go bigger!

- **Activation function**

- Classification: Softmax
- Regression: ReLU (or one of its variants)

- **Batch size**

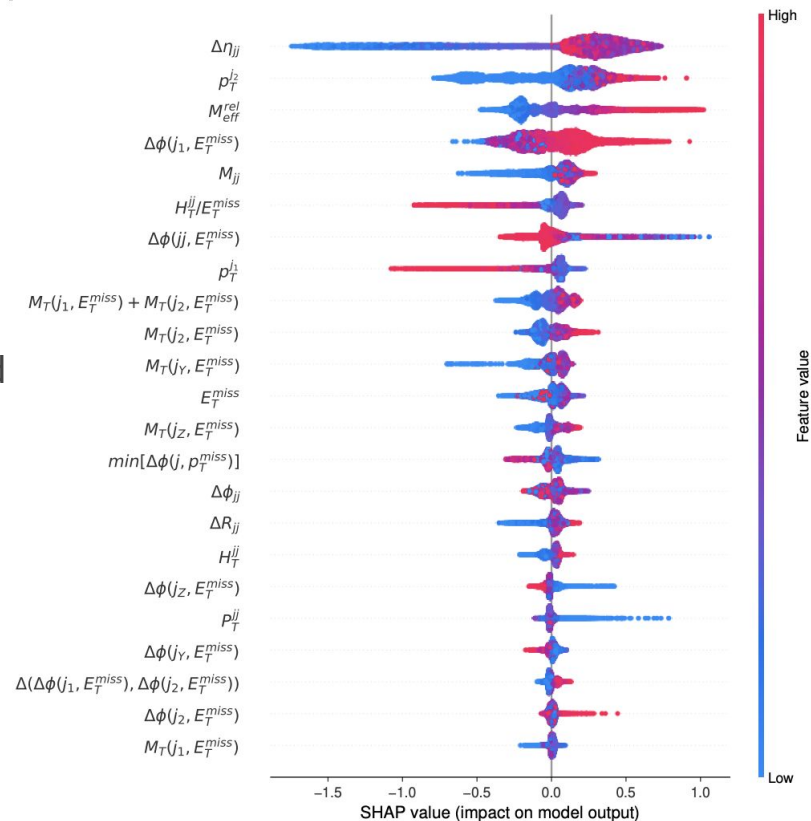
- Too large = faster training (with a GPU); better gradient estimate (= faster convergence but can get stuck at local minima)
- Too small = needs less memory but slower (more updates needed per epoch), “noisier” gradient descent (can help escape local minima or cause instability)
- Reasonable values to try initially: 32 / 64 / 128 / 256 / 512

- **Number of epochs / early stopping**

- Depends how fast the training converges

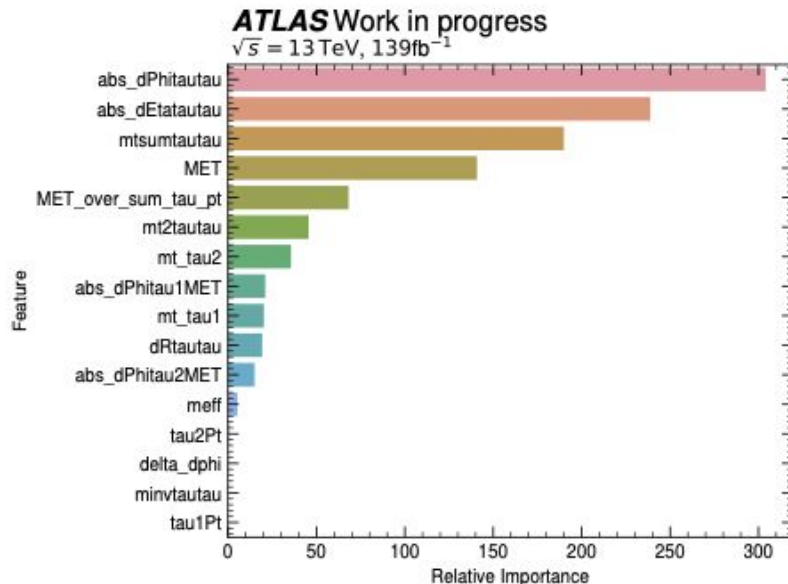
Interpretability

- What is your ML model doing under the hood?
- Some ways to do this:
 - Feature rankings
 - SHAP values - shows impact of feature values on model score
 - 2D plots of variables against classifier scores to understand correlations
 - Plot distributions of input variables in high score regions



Input features

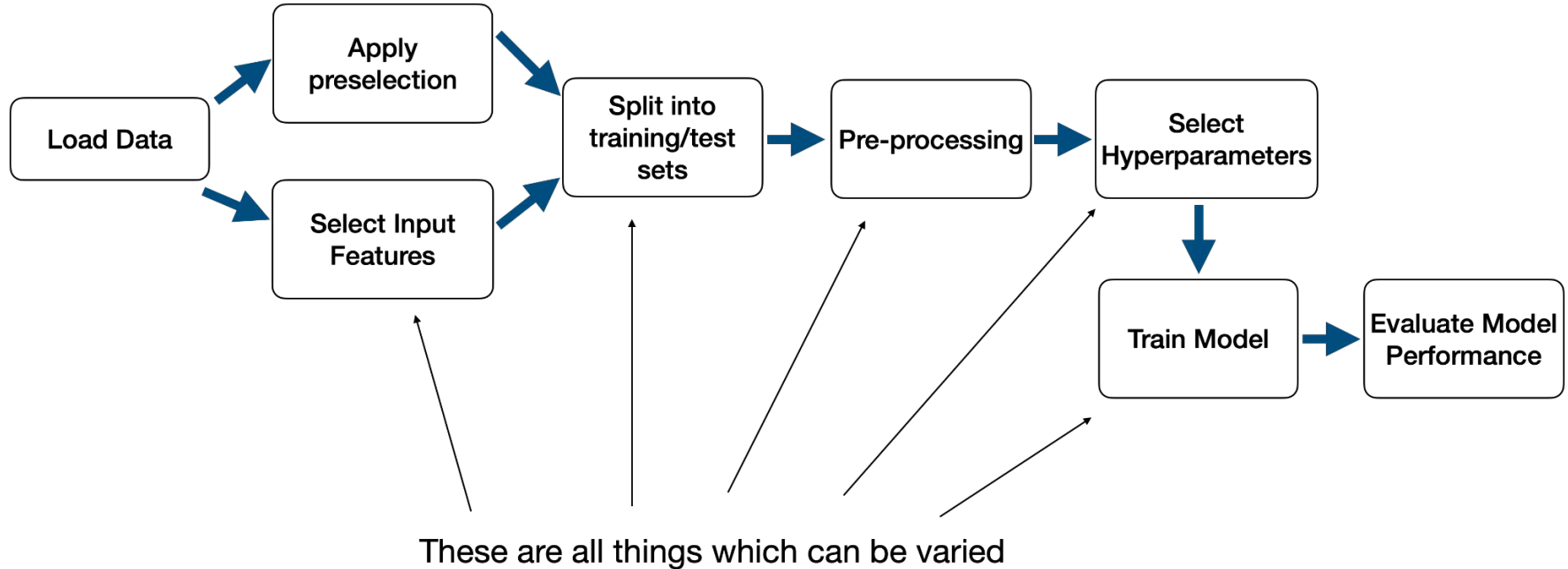
- High level (e.g. invariant mass) or low level (e.g. calorimeter energy deposits)?
 - **Think what's useful** for your use case!
- Feature pruning
 - **Remove features** with low importance to simplify models
 - **Remove variables that are very highly correlated** with others - not necessarily a problem but complicates interpretation
- Only use input features that are **well modelled (garbage in, garbage out)**
 - Also don't want to just learn generator differences



Here Relative Importance = how often feature used x gain when used

Note: NNs can easily add/subtract input features together, so don't need to provide e.g. the sum of two variables as a separate feature

Training And Optimising the Model



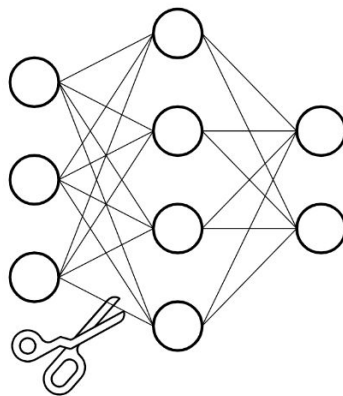
Regularization and Pruning

Regularization

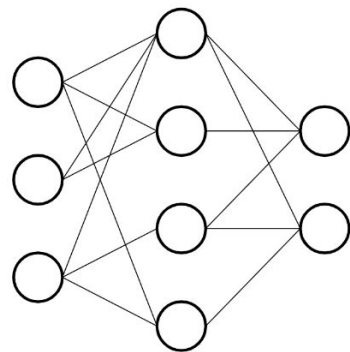
- Reduce model complexity by adding additional terms to the loss function
 - L1 Regularization (Lasso): Encourages sparsity by adding the sum of the absolute values of the parameters to the loss function.
 - L2 Regularization (Ridge): Penalizes large parameter values by adding the sum of squared parameters to the loss function.
- **Dropout** for NNs also help reduce overfitting → randomly set a fraction of the neurons to 0 during each forward pass of training

Pruning/Model Compression

- Remove unnecessary complexity of model without impacting performance
- E.g. removing basically redundant weights/neurons (NNs) or leaf nodes (BDTs)
- Can be done during/after model training
- Useful if minimising evaluation time is important for you
- See also “Lottery ticket hypothesis” for further reading



Before pruning



After pruning

What algorithm should I use?

No free lunch theorem

- *Any two optimization algorithms are equivalent when their performance is averaged across all possible problems*
- *I.e. No algorithm will work best for every supervised learning problem*

Considerations

- More advanced/complex model = better performance but more resource intensive
 - For **similar performance levels, simple model >> more complex model**
- How much **time** do you have for R&D?
- What **computing resources** do you have?
- How many **statistics** do you have? Can you request more?
- What has been **previously done**?
- What **'expertise'** do you have or have access to in your team/institute? (N.B. real expertise comes from actually using these methods)

Rough statistics guide:

- Basic BDT ~ 1000s-10000s events
- More complex BDT/simple NN ~ 10,000s-100,000s events
- DNN/GNN - 1,000,000s + events

For reference, GN2 ~200 Million input jets for training

What Software Package should I use?

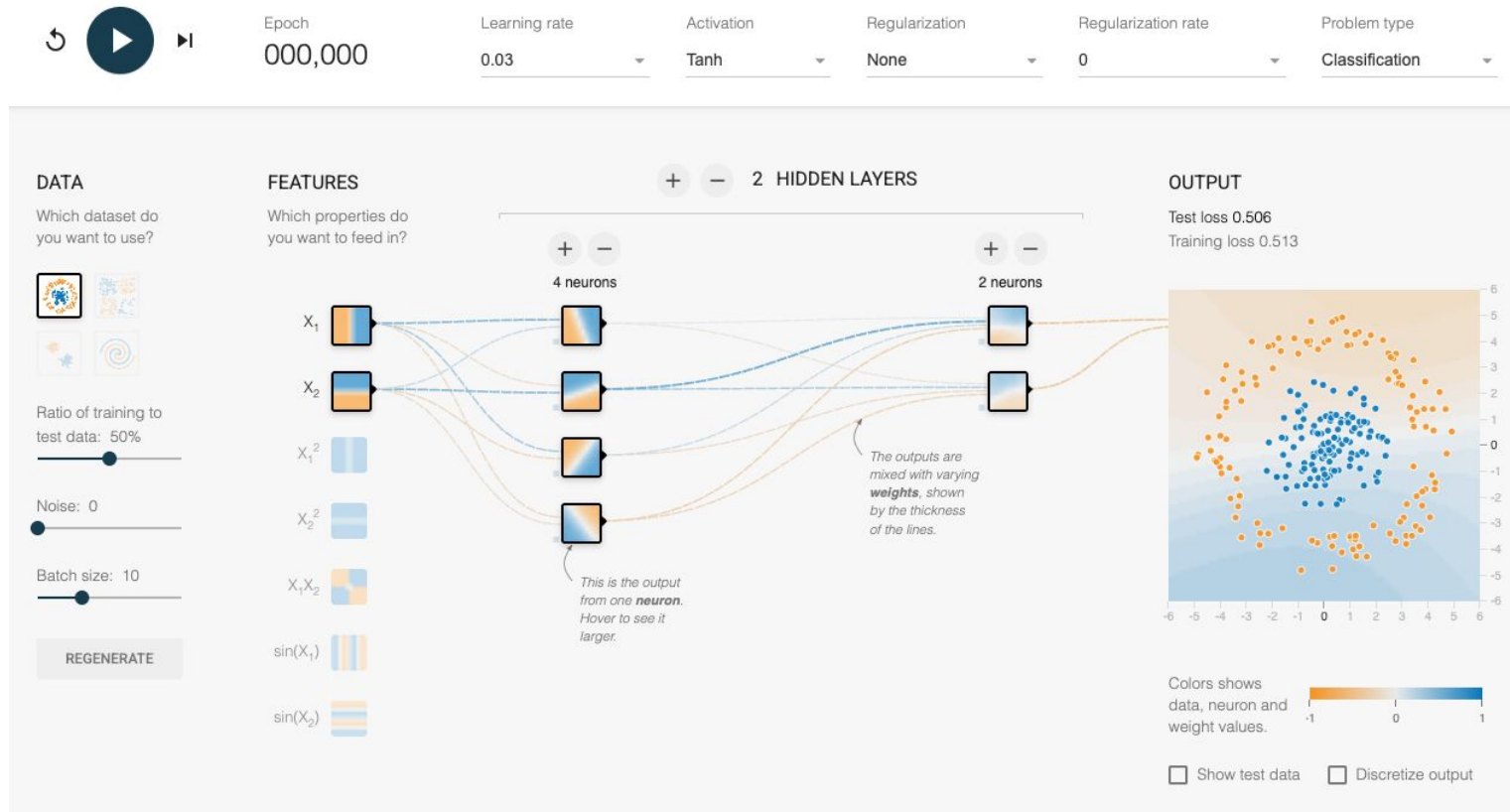
- For these different algorithms, several different software packages exist
- BDTs: [XGBoost](#), [LightGBM](#), [CatBoost](#)
- Neural Networks: [TensorFlow](#), [PyTorch](#), [Keras](#) (high level API)
- [SciKit learn](#) → useful intro material for simpler architectures
- ROOT TMVA → Generally would not recommend unless it's already a fundamental part of your analysis pipeline (less features, not used outside of HEP)
- For choosing a specific package, probably makes sense to go with what expertise you have access to in your group/analysis team

Things we haven't covered

- Unsupervised learning
 - Clustering
 - Generative models: GAN, VAE, Flow
- Convolutional neural networks (see backup slides)
 - Useful for image tasks (eg. images of detector hits)
- Transformers
- Recurrent Neural Networks (RNNs)
 - Sequential data (text, speech, etc.)
- Recursive Neural Networks
 - For Natural Language Processing
- Graph Neural Networks
 - To perform inference on data described by graphs
 - Ask Holly for tips!

TensorFlow Playground

<https://playground.tensorflow.org/>



Summary

- Introduced the basics of ML and some practical tips
- Lots more to learn about if you're interested!
- HEP is ripe for ML applications
- The tools are easier to use/more streamlined than ever before
- Have fun!

Hands-on

Use the “pythonic” (not ROOT) environment:
[README/#the-pythonic-tutorials](#)

1) [LHCB_classification.ipynb](#)

- LightGBM BDT example
- Includes examples/exercises related to preprocessing, quality metrics, feature importance, hyperparameter optimization, cross-validation

2) [METNet_example.ipynb](#)

- Keras neural network regression example
- Includes examples/exercises related to data cleaning, regression metrics, hyperparameter optimization & regularisation

Extra: [SimpleExample.ipynb](#) & [DecisionTreeDepth.ipynb](#)

- Example of a simple scikit-learn decision tree & demonstrates the impact of tree depth

Backup

BDTs – boosting, pruning and bagging

Boosting:

- Instead of building one complex tree a “forest” (often many hundred) of smaller trees is built
- Events are re-weighted according to how wrongly they were classified in the previous trees, i.e. wrongly classified events become more important → many ways - in most machine-learning packages steerable - to determine this weight (mean squared error loss, Huber loss, etc.)
 - Common boosting algorithms are adaptive and gradient boosting
- The final score is the weighted average across all trees in the forest → output is a float

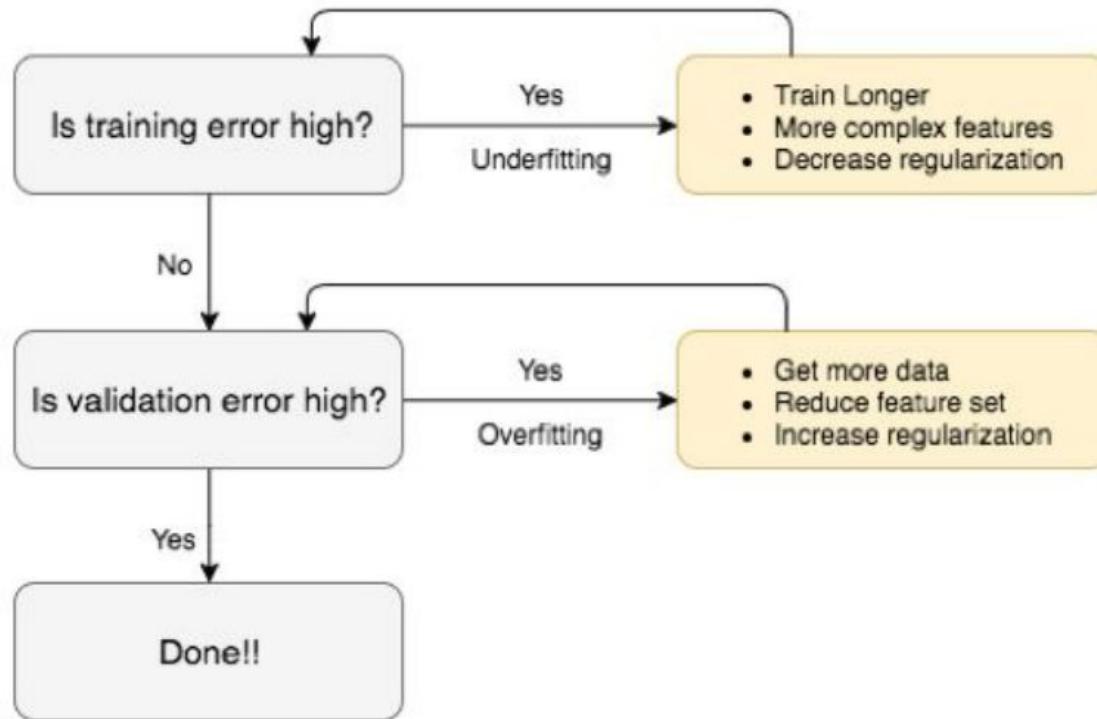
Pruning: Removal of non-significant nodes

Bagging: Retraining of the decision tree multiple times with random subset of the training data and an average of the output of all retrained trees is taken

Preselection - i.e. what Events to use in the Training

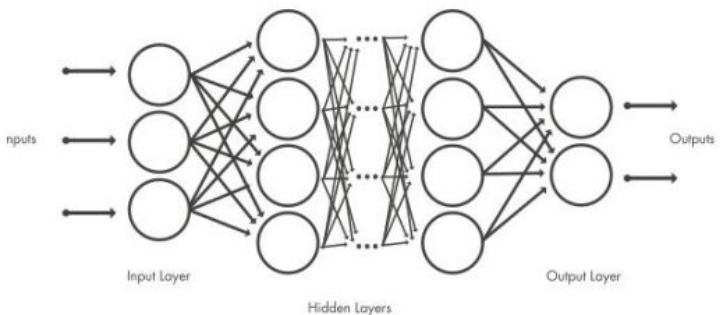
- General idea: **Use as much data as possible in the training** - the algorithms can likely better use the information than you can
- However there might be some **obvious cuts** you can apply
 - e.g dilepton invariant mass cut around Z-peak)
 - Allows algorithm to **'focus' on more challenging region of phase space** - can have a simpler model with same performance
 - Applying obvious cuts might also **make performance metrics more useful** (otherwise AUC scores etc. very high)
- Only subset of available phase space might be well modelled by MC - **Garbage in, garbage out**
- Might be useful to **fake objects/not provide information to improve stats**
 - E.g. Might be applying b-jet veto in final selection, but by not having it in the training you can improve statistics for top related backgrounds (useful provided this selection doesn't greatly alter the shapes of other kinematic variables you're using)

Training convergence flow chart

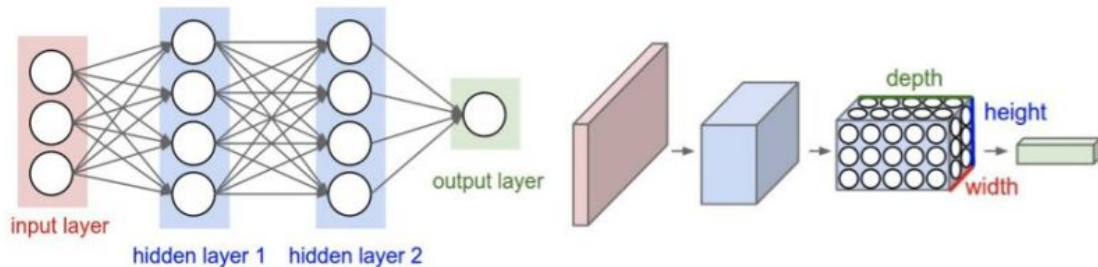


Convolutional neural networks (CNNs)

It is desirable to be able to *handle arbitrary-sized and -shaped data*

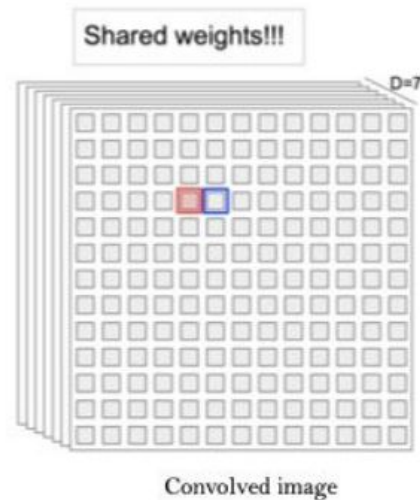
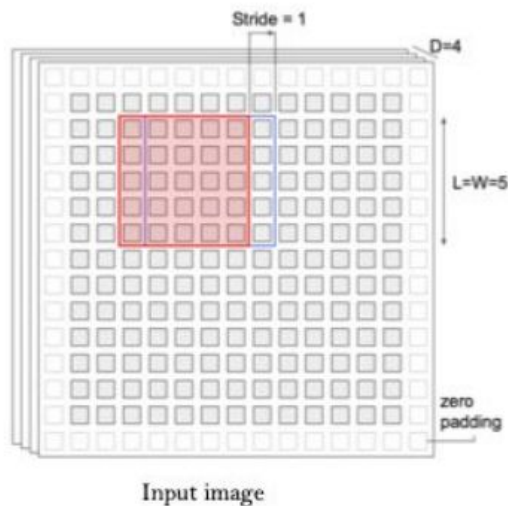
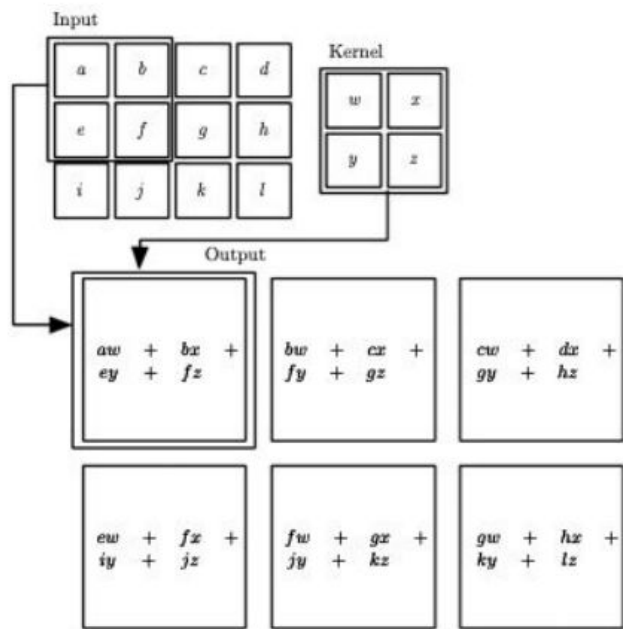


Convolutional networks (ConvNets) are aiming to do exactly that by relying on *convolution* operations



Convolutional neural networks (CNNs)

The convolution operation "smoothens" an *input* by **convoluting** it with a *kernel* to obtain a *feature map*.



Convolutional neural networks (CNNs)

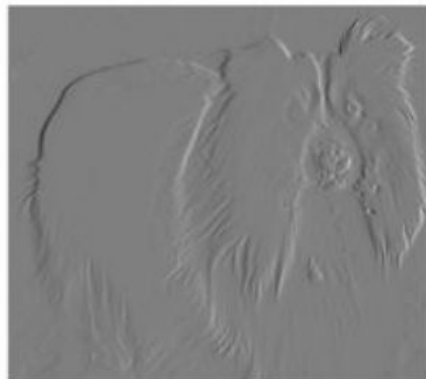
Feature detection



Input

1	-1
---	----

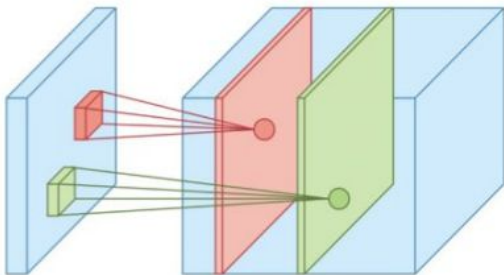
Kernel



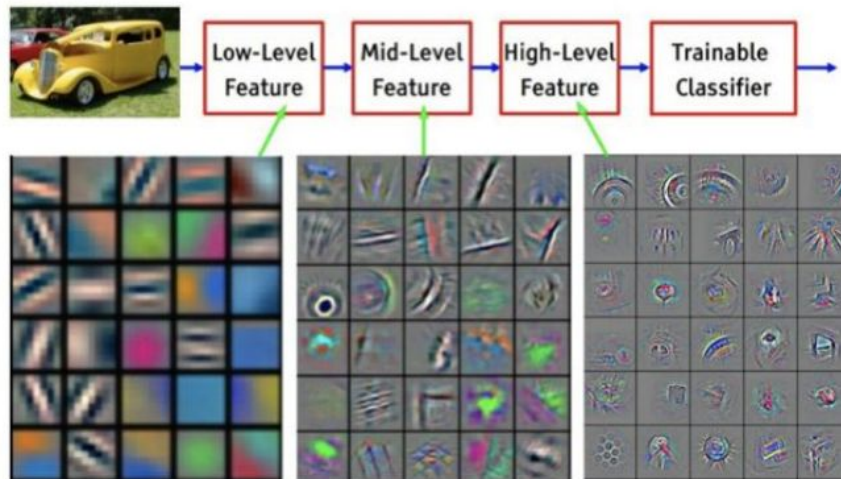
Output

Convolutional neural networks (CNNs)

Filter examples



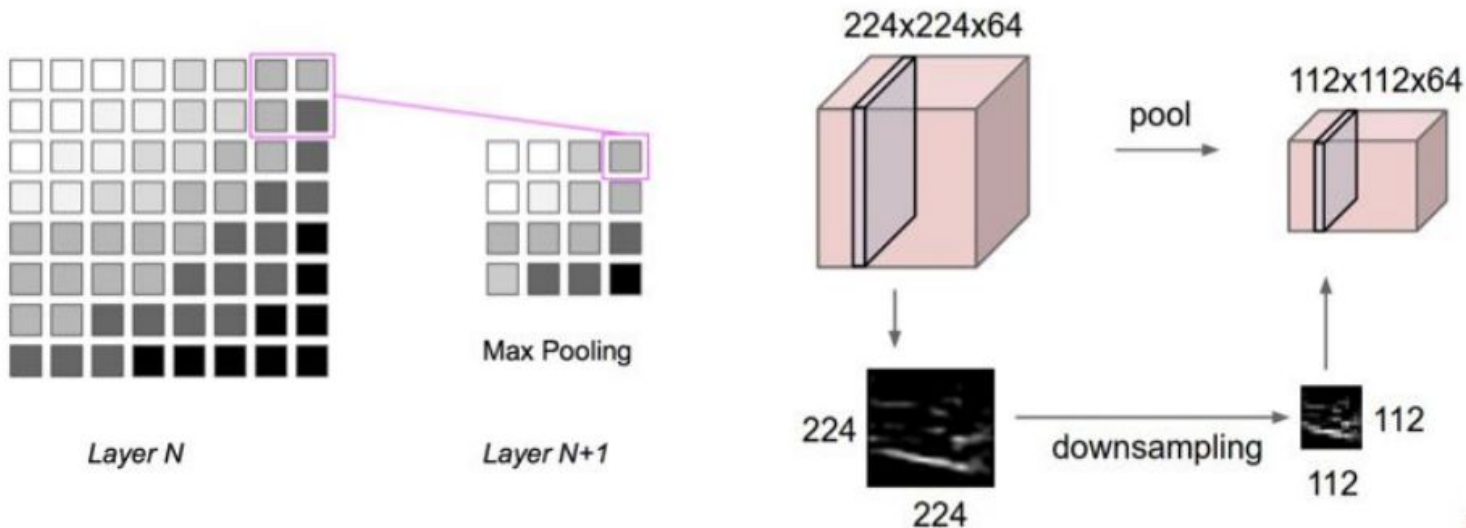
A convolution step can aim at detecting multiple features (shown as an extra dimension when representing CNNs)



Max Pooling

So far, it seems that we're *increasing the amount of data our network handles*, how are we getting any closer to a (small) set of final numbers?

Max Pooling down-samples the input by taking max or average over a region of inputs to *keep only the most useful information*



ConvNet Example

